Chapter 9: Conditional Statements

Structured Study Notes | python-zero-to-projects

Putting Operators to Work

In Chapter 3 you learned comparison operators (==, !=, >, <, >=, <=) and logical operators (and, or, not). At the time, they could produce True or False but had nowhere to go. Chapter 9 changes that: **conditional statements are the mechanism that uses those boolean results to control program flow** - deciding which block of code executes and which is skipped.

This chapter also unifies the data structures from Chapters 4-8: strings, lists, tuples, sets, and dictionaries all have a boolean interpretation (truthy/falsy) that you will use directly in conditions.

1. if, elif, and else Statements

1a. Basic if Statement

The simplest conditional runs a block of code only when a condition is True. The body must be indented (4 spaces by convention):

```
temperature = 35

if temperature > 30:
    print("It is hot outside!")   # runs only when condition is True

print("This always runs")         # outside the if block
```

1b. if-else: Two-Way Branch

else provides the code to run when the condition is False. Exactly one of the two branches always executes:

```
age = 16

if age >= 18:
    print("You can vote.")
else:
    print("You cannot vote yet.")

# Output: You cannot vote yet.
```

1c. if-elif-else: Multiple Branches (Grading System)

elif (short for "else if") lets you test multiple conditions in sequence. Python checks them top to bottom and runs the FIRST branch whose condition is True, then skips all remaining branches:

```
score = int(input("Enter your score (0-100): "))

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"

print("Grade:", grade)
```

Key point: once a condition matches, the rest of the chain is skipped. Order matters - putting score >= 60 first would catch everything above 60 before the more specific conditions could run.

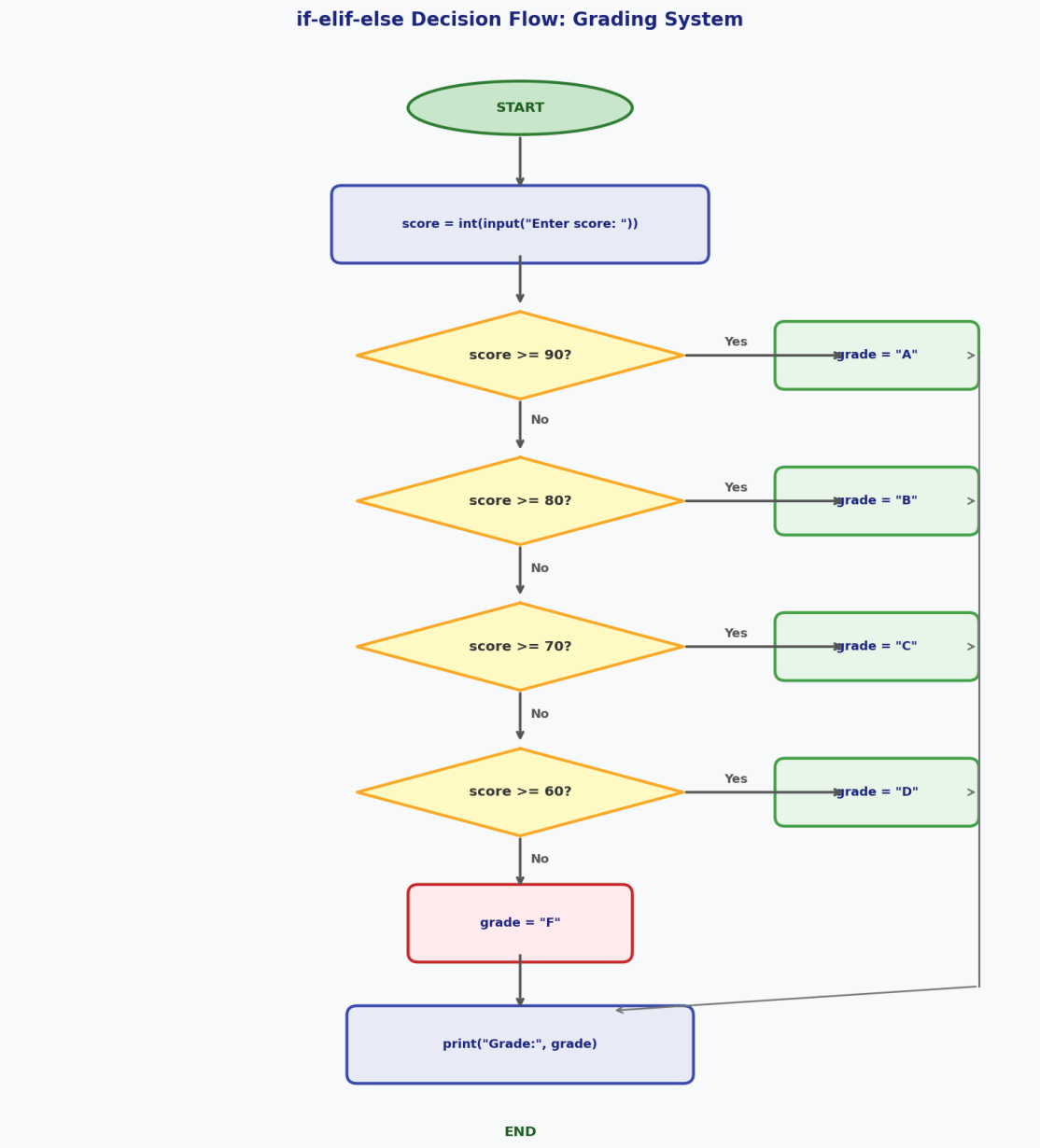

Figure 1: Flowchart showing the if-elif-else grading system. Python evaluates conditions top to bottom and executes only the first matching branch.

2. Nested Conditional Statements

A conditional statement can contain another conditional statement inside its body. This is called **nesting**. In Python, indentation determines which level a block belongs to - each nesting level adds 4 more spaces:

```
# Club entry checker: age AND ID AND membership
age = int(input("Enter age: "))
has_id = input("Do you have ID? (yes/no): ").lower() == "yes"
is_member = input("Are you a member? (yes/no): ").lower() == "yes"

if age >= 18:                       # Level 1 (4 spaces)
```

```
    if has_id:                          # Level 2 (8 spaces)
        if is_member:                   # Level 3 (12 spaces)
            print("Welcome!")
        else:
            print("Pay entry: $10")
    else:
        print("Please bring ID.")
else:
    print("Sorry, you must be 18 or older.")
```

WARNING: Deep nesting (more than 2-3 levels) makes code hard to read. As you learn more Python, you will learn techniques like early returns and combining conditions with and/or to flatten deeply nested code.

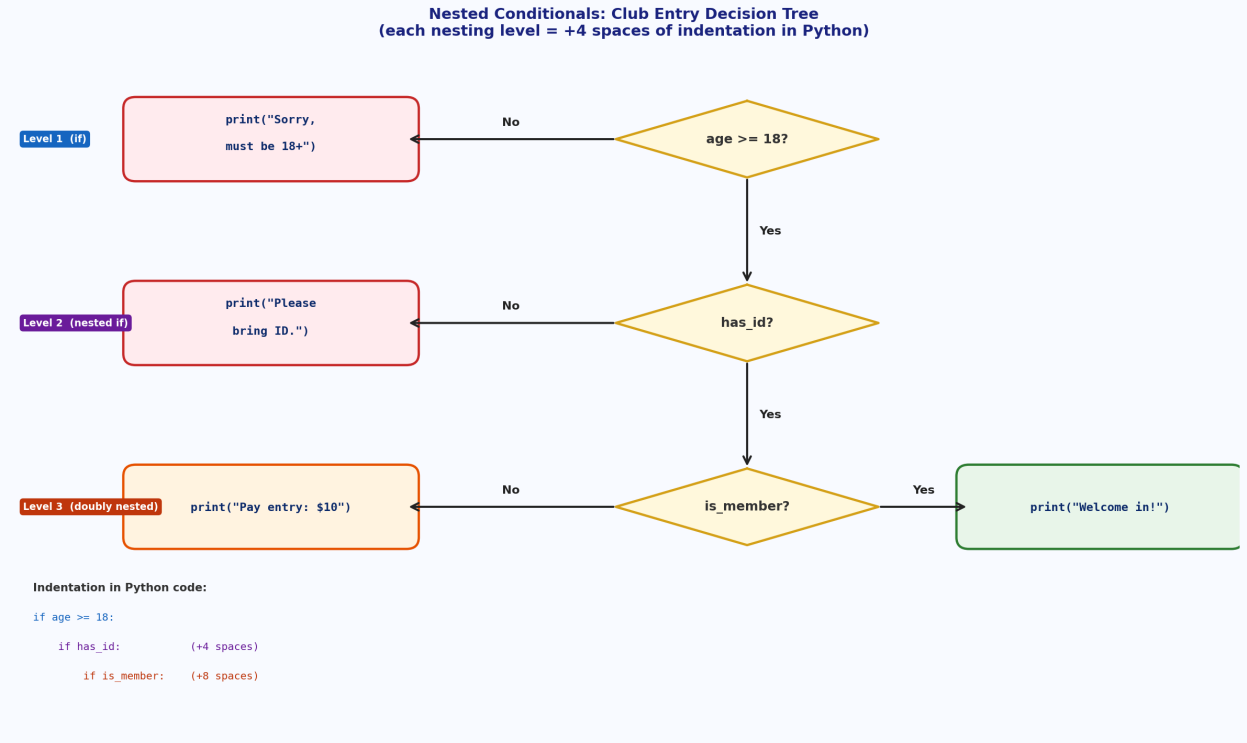

Figure 2: Decision tree for the nested club entry example. Each level of the tree corresponds to one deeper level of indentation in Python.

3. Truthy and Falsy Values

Every Python object has a boolean interpretation - it evaluates to either True or False when used in a condition. Objects that evaluate to False are called **falsy**; everything else is **truthy**.

The complete list of falsy values in Python:

```
# All of these evaluate to False in a condition:
False          # the boolean False
0              # integer zero
0.0            # float zero
""             # empty string   (Chapter 4)
[]             # empty list     (Chapter 5)
```

```
()             # empty tuple    (Chapter 6)
set()          # empty set      (Chapter 7)
{}             # empty dict     (Chapter 8)
None           # the None object

# Everything else is truthy:
# True, 1, -5, 3.14, "hi", [0], (False,), {1}, {"a":1}, any object
```

Synthesis across Chapters 4-8: Notice that an empty container from **every** data structure you have learned is falsy. Use it to write cleaner code:

```
# -- Chapter 4: Strings --
name = ""
if name:                          # Pythonic: falsy if empty
    print("Hello,", name)
else:
    print("Name is empty")

# -- Chapter 5: Lists --
my_list = []
if my_list:                       # Pythonic: falsy if empty
    print("List has items")

# -- Chapter 6: Tuples --
coords = ()
if coords:
    print("Has coordinates")

# -- Chapter 7: Sets --
unique_items = set()
if unique_items:
    print("Set has items")

# -- Chapter 8: Dictionaries --
config = {}
if config:
    print("Config is loaded")
else:
    print("Config is empty")
```

The Pythonic idiom: write "if my_list:" instead of "if len(my_list) != 0:". Both work, but the first is cleaner, shorter, and considered better style in Python.

Truthy and Falsy Values in Python

FALSY VALUES	Description
`False`	The boolean False
`0`	Integer zero
`0.0`	Float zero
`""`	Empty string (Ch. 4)
`[]`	Empty list (Ch. 5)
`()`	Empty tuple (Ch. 6)
`set()`	Empty set (Ch. 7)
`{}`	Empty dict (Ch. 8)
`None`	The None object

TRUTHY VALUES	Description
`True`	The boolean True
`1, -5, 42`	Any non-zero integer
`3.14, -0.001`	Any non-zero float
`"hi", " "`	Any non-empty string
`[0], [False]`	Any non-empty list
`(0,)`	Any non-empty tuple
`{1,2}`	Any non-empty set
`{"a":1}`	Any non-empty dict
`object refs`	Any object (by default)

Figure 3: Complete reference table of falsy values (red, left) and representative truthy values (green, right), with chapter references for the empty container types.

4. None and the is Operator

None is Python's way of representing "no value" or "nothing here". It is its own type (NoneType) and there is exactly one None object in any Python program:

```
result = None
print(result)          # None
print(type(result))    # <class 'NoneType'>

# Functions with no return statement return None
def greet(name):
    print("Hello,", name)

x = greet("Alice")     # prints "Hello, Alice"
print(x)               # None
```

Recalling Chapter 8: dict.get() and None

In Chapter 8 you saw that dict.get(key) returns None when the key does not exist instead of raising a KeyError. Checking for None is the standard way to handle this:

```
user_data = {"name": "Alice", "age": 30}

email = user_data.get("email")   # key missing -> returns None

if email is not None:
    print("Email:", email)
else:
    print("No email on file")
```

Use "is None" / "is not None", not "== None"

Python convention is to use the identity operator **is** when comparing to None. Since there is only one None object, is None is both more correct and more efficient:

```
# PREFERRED:
if result is None:
    print("No result")

if result is not None:
    print("Got a result:", result)

# AVOID (works but not idiomatic):
if result == None:     # style checkers will warn you
    print("No result")

# WHY: a custom class could define __eq__ to make (obj == None) True
# even when obj is not None. "is None" is always safe.
```

5. Short-Circuit Evaluation and Chained Comparisons

5a. Short-Circuit Evaluation

Python's and and or operators stop ("short-circuit") as soon as the result is certain:

```
# "and" short-circuits on the FIRST False operand
x = 0
if x != 0 and (10 / x) > 2:   # safe! division never runs when x == 0
    print("Result is large")

# "or" short-circuits on the FIRST True operand
name = "Alice"
if name or expensive_check():  # expensive_check() never called
    print("We have a name")
```

The "or default value" Pattern

Because or returns the first truthy value it finds, it is commonly used to assign a default:

```
user_input = input("Enter your name (or press Enter): ")
name = user_input or "Anonymous"
print("Hello,", name)

# More examples:
port  = user_config.get("port") or 8080    # default port
title = provided_title or "Untitled"       # default title

# "" or "Anonymous"  ->  "Anonymous"  (empty string is falsy)
# "Bob" or "Anonymous"  ->  "Bob"      (non-empty string is truthy)
```

5b. Chained Comparisons

Python allows you to chain comparison operators in a natural, mathematical style:

```
x = 5

# Traditional form (Chapter 3 style):
if x > 0 and x < 10:
    print("x is between 0 and 10")

# Chained form (more Pythonic):
if 0 < x < 10:
    print("x is between 0 and 10")    # same result, cleaner

score = 75
if 60 <= score <= 79:
    print("Grade: C")
```

6. Ternary / Conditional Expressions

Python's **ternary expression** lets you write a simple if-else as a single line:

```
# Full syntax:
# value_if_true  if  condition  else  value_if_false

# Multi-line if-else:
age = 20
if age >= 18:
    status = "adult"
else:
    status = "minor"

# Same using ternary - one line!
status = "adult" if age >= 18 else "minor"
print(status)   # "adult"
```

```
# Example 2: pass/fail
score = 72
result = "Pass" if score >= 60 else "Fail"
print(result)   # "Pass"

# Example 3: absolute value
num = -7
abs_val = num if num >= 0 else -num
print(abs_val)   # 7

# Example 4: inline default (preview of Chapter 11)
name = ""
print("Hello,", name if name else "Guest")   # "Hello, Guest"
```

Keep ternary expressions simple. Nested ternaries are legal but hard to read - avoid them.

7. The match Statement (Python 3.10+)

Python 3.10 introduced **structural pattern matching** via the match statement. It is most useful when you need to dispatch based on the value of a single variable.

7a. Basic Syntax: Matching Literal Values

```
command = input("Enter command: ")

match command:
    case "quit":
        print("Quitting the program...")
    case "help":
        print("Showing help menu")
    case "start":
        print("Starting the process...")
    case "stop":
        print("Stopping the process...")
    case _:
        print("Unknown command:", command)
```

7b. The Wildcard Case: _

The underscore _ is the catch-all wildcard, equivalent to the else clause. Always make it last:

```
match status_code:
    case 200:
        print("OK")
    case 404:
        print("Not Found")
    case 500:
        print("Server Error")
    case _:
        print("Unexpected code:", status_code)
```

7c. Matching Multiple Values with |

Use the | operator (OR) inside a case to match multiple literal values in a single branch:

```
match day.lower():
    case "saturday" | "sunday":
        print("Weekend - no work!")
    case "monday" | "friday":
        print("Start or end of the work week")
    case "tuesday" | "wednesday" | "thursday":
        print("Midweek")
    case _:
        print("Not a valid day")
```

NOTE: The | inside a match-case means "or this pattern" - it does not perform arithmetic like the bitwise OR from Chapter 3 or set union from Chapter 7.

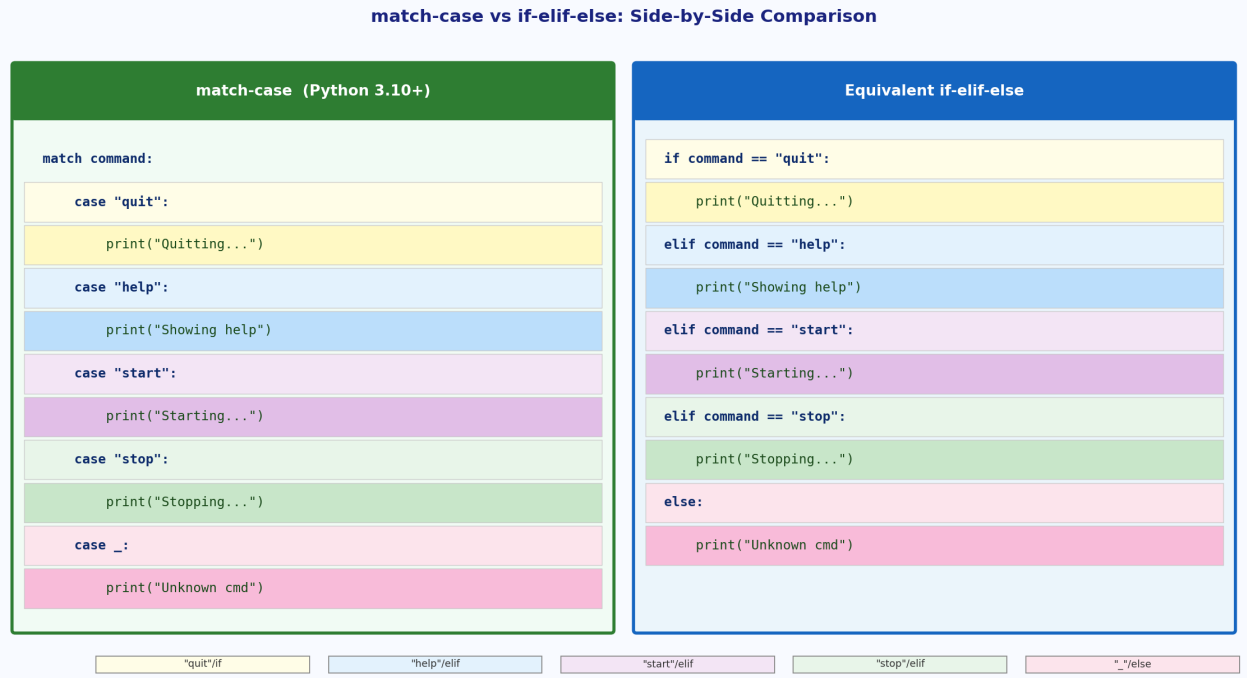

Figure 4: Side-by-side comparison of a match-case block (left) and its equivalent if-elif-else chain (right). Matching colour bands show which case corresponds to which branch.

Common Beginner Mistakes with Conditionals

Mistake 1: Using = (assignment) instead of == (comparison) in a condition

```
# WRONG: = is assignment - Python raises SyntaxError here
# if x = 5:   <- SyntaxError

# CORRECT: use == for equality testing
x = 10
if x == 10:
    print("x is ten")
```

Mistake 2: Using separate if statements when elif was intended

```
# WRONG: all conditions checked independently
score = 85
if score >= 90:
    print("A")    # skipped
if score >= 80:
    print("B")    # prints "B"
if score >= 70:
    print("C")    # ALSO prints "C" - two grades!

# CORRECT: use elif so only first match runs
if score >= 90:
    print("A")
elif score >= 80:
```

```
    print("B")    # prints "B" and stops
```

Mistake 3: Using == None instead of is None

```
# AVOID:
if result == None:      # works but not idiomatic
    print("no result")

# PREFERRED:
if result is None:
    print("no result")
if result is not None:
    print("got a result")
```

Mistake 4: Indentation errors that silently change program logic

```
# WRONG: print is outside the if block
score = 55
if score >= 60:
    print("You passed!")
print("Certificate issued")   # runs ALWAYS!

# CORRECT:
if score >= 60:
    print("You passed!")
    print("Certificate issued")   # only runs when score >= 60
```

Mistake 5: Forgetting the wildcard _ in a match statement

```
# Without _, unmatched values silently do nothing:
command = "restart"
match command:
    case "start":
        print("Starting")
    case "stop":
        print("Stopping")
# "restart" matches nothing - no output, no error

# CORRECT: always add _ as last case:
    case _:
        print("Unknown command:", command)
```

Mistake 6: "if len(my_list) != 0:" instead of the Pythonic "if my_list:"

```
# VERBOSE (not wrong, but un-Pythonic):
if len(my_list) != 0:
    print("List has items")

# PYTHONIC:
```

```
if my_list:
    print("List has items")

# Same applies to strings, tuples, sets, dicts:
# if my_string:   if my_dict:   if my_set:
```

Preview: Chapters 10 and 11

Everything you learned in this chapter feeds directly into the next two chapters:

- Chapter 10 (Loops): while loops run a block repeatedly **as long as a condition is True**. The break and continue statements inside loops almost always depend on an inner if statement.
- Chapter 11 (Functions): functions frequently use if-elif-else to decide what to return. The ternary expression is very common inside short functions as a one-line return statement.

Chapter 9 Summary and Recap

- Conditional statements use the comparison and logical operators from Chapter 3 to control which code runs.
- if runs a block when True; if-else gives a two-way branch; if-elif-else chains test multiple conditions in order, running only the first match.
- Nested conditionals place one if inside another; each nesting level adds 4 spaces of indentation.
- Falsy values: False, 0, 0.0, "", [], (), set(), {}, None. All empty containers (Chs. 4-8) are falsy. Everything else is truthy.
- Write "if my_list:" not "if len(my_list) != 0:" - use truthy/falsy behavior directly.
- None represents "no value". Use "is None" / "is not None" (not == None).
- and and or short-circuit: they stop as soon as the result is certain. Use "value = x or default" for fallback values.
- Chained comparisons: "0 < x < 10" is cleaner than "x > 0 and x < 10".
- Ternary expression: "value_if_true if condition else value_if_false" - one-line conditional assignment.
- match statement (Python 3.10+): cleaner than long if-elif chains. Use _ as the wildcard; | matches multiple values in one case.
- Common pitfalls: = vs ==, separate ifs vs elif, == None vs is None, indentation errors, missing _ in match, verbose len() checks.

--- End of Chapter 9 Notes ---